```
#include <iostream>
using namespace std;
void change(int &a) {
    a=100;
}
int main () {
    int x = 10;
    change(x);
    cout << x ; # 100
    return 0;
}
```

Student marks modifier using call by reference.

Day-9 (Arrays And Pointer Arithmetic)

An array is a collection of elements of the same data type stored in continuous memory location

[10][20][30][40][50]...
100 104 108 112 ... (Because int take 4 byte space)

All stored side by side in memory.

```
int arr[5] = {10, 20, 30, 40, 50}
```

Note:
- Each Elements take 4 bytes.
- Address increases by 4
- Stored in continuous memory.

cessing Array Element.

```
cout << arr[0] ;   //10
cout << arr[1] ;   //20
```

Index start from 0, not 1.

Problem statement: Array Analyzer using Pointer Arithmetic.

formula internally:

arr[i] = *(arr + i)

this is where pointer arithmatics starts.

Array and pointers relationship

when you write:

int arr[3] = {1,2,3};

The name arr itself act like a pointer to the first element

arr == &arr[0]

that means

cout << *arr; //1

cout << *(arr+1); //2

cout << *(arr+2); //3

Pointers Arithmetics.

what happen when we do (ptr+1)

it does'nt mean it move by 1 byte.

it move by size of data type

for int, float → 4 byte

for ~~bool~~ double → 8 byte.

for char → 1 bytes.

why pointer Arithmatics works in array?

because arrays are stored in continuous memory.

that's why:

arr[i] = *(arr + i)

this is how compiler actually calculates index internal

difference:

arrays	pointers.
fixed size	can move.
name cannot be changed.	pointers can be change.
arr++ ✗ not allowed.	ptr++ allowed.